Matrix-free preconditioned conjugate gradients for an RKHS-constrained CP mode update with missing data

Abstract

We explain how to solve, in a matrix-free way, the normal equations arising in the mode-k subproblem of a CP decomposition with missing entries when the mode-k factor is constrained to lie in an RKHS. The linear system has size $(nr) \times (nr)$, where $n = n_k$ and r is the CP rank, and naïve direct solvers scale as $O(n^3 r^3)$ (and require explicit assembly of large Kronecker/Khatri–Rao objects). We show how to apply the system matrix to a vector without forming any object of size $N = \prod_i n_i$ or $M = \prod_{i \neq k} n_i$ by exploiting (i) the selection/masking operator for missing data, (ii) Kronecker identities, and (iii) the Khatri–Rao structure evaluated only on the $q \ll N$ observed entries. This enables preconditioned conjugate gradients (PCG) with per-iteration cost $O(qr + n^2 r)$ (assuming a dense kernel matrix K), plus a preconditioner application typically of cost $O(n^2 r)$ after one-time factorizations.

1 Problem and notation

Let $\mathcal{T} \in \mathbb{R}^{n_1 \times \cdots \times n_d}$ be a d-way tensor with missing entries. Fix a mode $k \in \{1, \ldots, d\}$ and denote

$$n \equiv n_k, \qquad M \equiv \prod_{i \neq k} n_i, \qquad N = nM.$$

Let $T \in \mathbb{R}^{n \times M}$ be the mode-k unfolding of $\mathcal{T}$, where missing entries are set to zero. Let $q \ll N$ denote the number of observed entries. Let $S \in \mathbb{R}^{N \times q}$ be the selection matrix so that $S^T \operatorname{vec}(T) \in \mathbb{R}^q$ extracts the observed entries from $\operatorname{vec}(T)$.

Let

$$Z = A_d \odot \cdots \odot A_{k+1} \odot A_{k-1} \odot \cdots \odot A_1 \in \mathbb{R}^{M \times r}$$

be the Khatri–Rao product of all factor matrices except mode k; the other factors are assumed fixed in the mode-k subproblem. Let $B = TZ \in \mathbb{R}^{n \times r}$ denote the (sparse) MTTKRP.

We assume the mode-k factor is constrained to an RKHS via

$$A_k = KW, \qquad K \in \mathbb{R}^{n \times n} \text{ PSD kernel matrix}, \qquad W \in \mathbb{R}^{n \times r} \text{ unknown}.$$

The linear system to be solved is

$$\Big[(Z \otimes K)^T S S^T (Z \otimes K) + \lambda (I_r \otimes K)\Big] \operatorname{vec}(W) = (I_r \otimes K) \operatorname{vec}(B), \tag{1}$$

where $\lambda > 0$ and I_r is the $r \times r$ identity.

A direct solver on (1) would require forming an $(nr) \times (nr)$ matrix and costs $O((nr)^3) = O(n^3 r^3)$ time (and $O(n^2 r^2)$ memory), which is prohibitive for moderately large n, r. We now show how to solve (1) using PCG, requiring only matrix-vector products and a preconditioner, both implemented without forming any object of size N or M.

2 Projection onto observed entries

Let $\Omega \subseteq [n] \times [M]$ denote the set of observed entries in the unfolding T, with $|\Omega| = q$. Define the masking (orthogonal projection) operator

$$\mathcal{P}_\Omega : \mathbb{R}^{n\times M} \to \mathbb{R}^{n\times M}, \qquad (\mathcal{P}_\Omega(X))_{ij} = \begin{cases} X_{ij}, & (i,j) \in \Omega, \\ 0, & \text{otherwise.} \end{cases}$$

Lemma 1 (Selection matrix as a mask)**.** *Let $S \in \mathbb{R}^{N\times q}$ select the entries in Ω from* $\mathrm{vec}(\cdot)$*. Then for any $X \in \mathbb{R}^{n\times M}$,*

$$SS^T \,\mathrm{vec}(X) = \mathrm{vec}(\mathcal{P}_\Omega(X)).$$

Proof. The selection matrix S consists of q distinct columns of the $N \times N$ identity, i.e. columns $\{e_\omega\}_{\omega\in\Omega}$ under the vectorization indexing. Hence

$$SS^T = \sum_{\omega\in\Omega} e_\omega e_\omega^T,$$

which is a diagonal projector that keeps entries in Ω and zeros the rest. Interpreting the result back as a matrix under vec^{-1} yields $\mathcal{P}_\Omega(X)$. □

3 Kronecker identities and an operator form

We use the standard identity

$$(B \otimes A)\,\mathrm{vec}(X) = \mathrm{vec}(AXB^T), \tag{2}$$

valid for conforming dimensions.

Define the linear operator $\mathcal{A} : \mathbb{R}^{n\times r} \to \mathbb{R}^{n\times r}$ by

$$\mathcal{A}(X) \;:=\; K\,\mathcal{P}_\Omega\big(KXZ^T\big)\,Z \;+\; \lambda KX. \tag{3}$$

Also define the right-hand side matrix

$$C := KB \in \mathbb{R}^{n\times r}. \tag{4}$$

Proposition 1 (Equivalence of the vector system and matrix operator form)**.** *Assume K is symmetric. Then* (1) *is equivalent to the matrix equation*

$$\mathcal{A}(W) = C. \tag{5}$$

Moreover, for any $X \in \mathbb{R}^{n\times r}$,

$$\mathrm{vec}(\mathcal{A}(X)) = \Big[(Z\otimes K)^T SS^T (Z\otimes K) + \lambda(I_r \otimes K)\Big]\,\mathrm{vec}(X).$$

Proof. Let $X \in \mathbb{R}^{n\times r}$. By (2),

$$(Z\otimes K)\,\mathrm{vec}(X) = \mathrm{vec}(KXZ^T).$$

Applying SS^T and Lemma 1 gives

$$SS^T(Z \otimes K)\,\mathrm{vec}(X) = \mathrm{vec}(\mathcal{P}_\Omega(KXZ^T)).$$

Now apply $(Z \otimes K)^T = (Z^T \otimes K^T)$ and (2) again:

$$(Z^T \otimes K^T)\,\mathrm{vec}(\mathcal{P}_\Omega(KXZ^T)) = \mathrm{vec}\Big(K^T\,\mathcal{P}_\Omega(KXZ^T)\,Z\Big).$$

Since K is symmetric, $K^T = K$. The regularization term satisfies

$$(I_r \otimes K)\,\mathrm{vec}(X) = \mathrm{vec}(KX).$$

Combining yields $\mathrm{vec}(\mathcal{A}(X))$ as claimed. Setting $X = W$ and using the right-hand side $(I_r \otimes K)\,\mathrm{vec}(B) = \mathrm{vec}(KB) = \mathrm{vec}(C)$ gives (5). □

Remark 1 (SPD conditions). The matrix in (1) is symmetric and positive semidefinite. If K is strictly positive definite and $\lambda > 0$, then the regularization term $\lambda(I_r \otimes K)$ is SPD and the whole system is SPD, so conjugate gradients applies. If K is only semidefinite, one may (i) add a small nugget $K \leftarrow K + \delta I$ or (ii) use MINRES on the symmetric semidefinite system. In what follows we assume SPD so that PCG is well-defined.

4 PCG without forming the $(nr) \times (nr)$ matrix

Conjugate gradients (CG) solves $Ax = b$ for SPD A using only matrix-vector products $v \mapsto Av$. Preconditioned CG (PCG) additionally uses an SPD preconditioner $M \approx A$ through solves $z = M^{-1}r$ in each iteration, accelerating convergence.

In our setting, $x = \mathrm{vec}(W) \in \mathbb{R}^{nr}$ and A is the bracketed matrix in (1). Proposition 1 shows that we can implement the matrix-vector product $y = Ax$ by applying the operator $\mathcal{A}$ to the reshaped matrix $X = \mathrm{vec}^{-1}(x) \in \mathbb{R}^{n\times r}$ and then vectorizing:

$$Ax = \mathrm{vec}(\mathcal{A}(X)).$$

We therefore implement PCG entirely in matrix form, using Frobenius inner products

$$\langle U, V\rangle_{\mathrm{F}} := \mathrm{tr}(U^T V) \qquad \text{so that} \qquad \langle \mathrm{vec}(U), \mathrm{vec}(V)\rangle = \langle U, V\rangle_{\mathrm{F}}\,.$$

4.1 PCG iteration (matrix form)

Let W_0 be an initial guess (e.g. $W_0 = 0$). Define $R_0 = C - \mathcal{A}(W_0)$. Given a preconditioner operator $\mathcal{M}^{-1}$ approximating $\mathcal{A}^{-1}$, set $Z_0 = \mathcal{M}^{-1}(R_0)$ and $P_0 = Z_0$. The standard PCG updates are:

The practical stopping criterion is based on the relative residual

$$\frac{\|R_t\|_{\mathrm{F}}}{\|C\|_{\mathrm{F}}} \le \varepsilon$$

for a desired tolerance ε (e.g. 10^{-6} to 10^{-3} depending on the outer alternating optimization).

Algorithm 1 Preconditioned conjugate gradients for (5)

Input: operator $\mathcal{A}$, right-hand side C, preconditioner $\mathcal{M}^{-1}$, initial W_0
$R_0 \leftarrow C - \mathcal{A}(W_0)$
$Z_0 \leftarrow \mathcal{M}^{-1}(R_0)$
$P_0 \leftarrow Z_0$
$\rho_0 \leftarrow \langle R_0, Z_0 \rangle_{\mathrm{F}}$
for $t = 0, 1, 2, \ldots$ until convergence **do**
  $Q_t \leftarrow \mathcal{A}(P_t)$
  $\alpha_t \leftarrow \rho_t / \langle P_t, Q_t \rangle_{\mathrm{F}}$
  $W_{t+1} \leftarrow W_t + \alpha_t P_t$
  $R_{t+1} \leftarrow R_t - \alpha_t Q_t$
  $Z_{t+1} \leftarrow \mathcal{M}^{-1}(R_{t+1})$
  $\rho_{t+1} \leftarrow \langle R_{t+1}, Z_{t+1} \rangle_{\mathrm{F}}$
  $\beta_t \leftarrow \rho_{t+1} / \rho_t$
  $P_{t+1} \leftarrow Z_{t+1} + \beta_t P_t$
end for
Output: W_{t+1}

5 Matrix-vector products in $O(qr + n^2 r)$ time

The key is to compute $\mathcal{A}(X) = K\,\mathcal{P}_\Omega(KXZ^T)\,Z + \lambda KX$ without forming any $n \times M$ dense matrix, nor $Z \in \mathbb{R}^{M \times r}$ explicitly.

5.1 Observed-entry representation and implicit rows of Z

Store the observed tensor entries as a list of q multi-indices and values:

$$\big\{(i_1^{(t)}, \ldots, i_d^{(t)}),\ \tau_t\big\}_{t=1}^{q},$$

where $\tau_t = \mathcal{T}_{i_1^{(t)}, \ldots, i_d^{(t)}}$ is the observed value. In terms of the mode-k unfolding, each observation corresponds to a pair

$$(i^{(t)}, j^{(t)}) \in [n] \times [M], \qquad \text{with } i^{(t)} = i_k^{(t)},$$

and $j^{(t)}$ the lexicographic index of the tuple $(i_1^{(t)}, \ldots, i_{k-1}^{(t)}, i_{k+1}^{(t)}, \ldots, i_d^{(t)})$.

For each observation t, define the *implicit* Khatri–Rao row vector

$$z^{(t)} \in \mathbb{R}^r, \qquad z^{(t)} = A_1(i_1^{(t)}, :) \odot \cdots \odot A_{k-1}(i_{k-1}^{(t)}, :) \odot A_{k+1}(i_{k+1}^{(t)}, :) \odot \cdots \odot A_d(i_d^{(t)}, :). \tag{6}$$

This is exactly the row $Z(j^{(t)}, :)$, but computed without forming Z. Since the factor matrices other than A_k are fixed throughout the mode-k subproblem, all vectors $\{z^{(t)}\}_{t=1}^{q}$ can be precomputed once and reused across all CG iterations (or computed on the fly if memory is tight).

5.2 A formula for the masked multiplication

Let $Y \in \mathbb{R}^{n \times r}$ and consider the (conceptual) product $YZ^T \in \mathbb{R}^{n \times M}$. For each observed entry $(i^{(t)}, j^{(t)}) \in \Omega$,

$$(YZ^T)_{i^{(t)}, j^{(t)}} = \Big\langle Y(i^{(t)}, :), Z(j^{(t)}, :) \Big\rangle = \Big\langle Y(i^{(t)}, :), z^{(t)} \Big\rangle.$$

Therefore, the masked matrix $\mathcal{P}_\Omega(YZ^T)$ has nonzeros only at the q observed positions, with values $\langle Y(i^{(t)},:), z^{(t)}\rangle$.

Now consider the matrix $(\mathcal{P}_\Omega(YZ^T))Z \in \mathbb{R}^{n\times r}$. Its ith row is

$$\big[(\mathcal{P}_\Omega(YZ^T))Z\big](i,:) = \sum_{t:\, i^{(t)}=i} \left(\left\langle Y(i,:), z^{(t)}\right\rangle\right) (z^{(t)})^T. \tag{7}$$

Equation (7) is the crucial identity: it shows we can compute $(\mathcal{P}_\Omega(YZ^T))Z$ by looping over the q observations, performing a length-r dot product and accumulating a length-r vector into row $i^{(t)}$.

5.3 Matrix-free evaluation of $\mathcal{A}(X)$

Given an input $X \in \mathbb{R}^{n\times r}$, compute $\mathcal{A}(X)$ as follows:

1. Compute $Y \leftarrow KX$ (dense kernel multiply).
2. Compute $U \leftarrow (\mathcal{P}_\Omega(YZ^T))Z$ using (7): initialize $U = 0 \in \mathbb{R}^{n\times r}$ and for $t = 1, \ldots, q$ do
$$s_t \leftarrow \left\langle Y(i^{(t)},:), z^{(t)}\right\rangle \in \mathbb{R}, \qquad U(i^{(t)},:) \leftarrow U(i^{(t)},:) + s_t (z^{(t)})^T. \tag{8}$$
3. Return $\mathcal{A}(X) \leftarrow K(U + \lambda X)$.

This exactly matches (3) with $Y = KX$.

Proposition 2 (Correctness of the matrix-free product)**.** *The three-step procedure above returns $\mathcal{A}(X)$ as defined in* (3)*. Equivalently, it computes*

$$\mathrm{vec}(\mathcal{A}(X)) = \Big[(Z\otimes K)^T SS^T (Z\otimes K) + \lambda(I_r \otimes K)\Big]\,\mathrm{vec}(X)$$

without forming any object of size N or M.

Proof. Step 1 produces $Y = KX$. Step 2 implements (7), which is precisely the matrix product $(\mathcal{P}_\Omega(YZ^T))Z$. Step 3 multiplies by K on the left and adds the regularization contribution λKX, yielding exactly (3). □

5.4 Computing the right-hand side without forming Z or T

Although $B = TZ$ is assumed given in the prompt, it is instructive (and often necessary in practice) to compute it in the same sparse fashion, avoiding $O(nMr)$ work. Since T has zeros at missing entries,

$$B(i,:) = \sum_{(i,j)\in\Omega} T_{ij}\, Z(j,:).$$

In terms of the observation list, this becomes

$$B(i,:) = \sum_{t:\, i^{(t)}=i} \tau_t\, (z^{(t)})^T,$$

which can be computed in $O(qr)$ time by accumulation analogous to (8). Then $C = KB$ costs one dense multiplication by K.

6 Preconditioning

PCG requires a preconditioner M (or operator $\mathcal{M}$) that is:

- SPD,
- cheap to apply (i.e. solve $\mathcal{M}(X) = R$ efficiently),
- a reasonable approximation to $\mathcal{A}$ so that $\kappa(\mathcal{M}^{-1}\mathcal{A})$ is small.

We describe two practical choices that preserve the "no N" constraint.

6.1 Baseline: regularization (kernel) preconditioner

A simple and robust preconditioner is to keep only the Tikhonov term:

$$\mathcal{M}_0(X) := \lambda KX, \qquad \text{equivalently} \qquad M_0 := \lambda(I_r \otimes K). \tag{9}$$

Then applying $\mathcal{M}_0^{-1}$ amounts to solving K-systems:

$$\mathcal{M}_0^{-1}(R) = \frac{1}{\lambda} K^{-1} R.$$

If K is dense SPD, we can compute its Cholesky factorization $K = LL^T$ once (cost $O(n^3)$) and apply K^{-1} to r right-hand sides by two triangular solves per column (cost $O(n^2 r)$ per PCG iteration).

Why it helps. When the regularization is not extremely small, λK captures the dominant spectral scale of $\mathcal{A}$, and $\mathcal{M}_0^{-1}\mathcal{A}$ becomes closer to the identity. In particular,

$$\mathcal{M}_0^{-1}\mathcal{A}(X) = X + \frac{1}{\lambda} K^{-1}\Big(K\mathcal{P}_\Omega(KXZ^T)Z\Big) = X + \frac{1}{\lambda}\mathcal{P}_\Omega(KXZ^T)Z,$$

so the (preconditioned) perturbation away from X is scaled by $1/\lambda$.

6.2 A stronger Kronecker-style preconditioner using Gram information in r

One can incorporate coupling across the r components by approximating the data term in $\mathcal{A}$ with a separable (Kronecker) form. Define the small $r \times r$ Gram matrix

$$G := Z^T Z \in \mathbb{R}^{r\times r}. \tag{10}$$

Crucially, G can be computed *without forming* Z using the Khatri–Rao identity

$$Z^T Z =_{i\neq k} (A_i^T A_i), \tag{11}$$

where denotes the Hadamard (entrywise) product of the $r\times r$ matrices $A_i^T A_i$. This costs $O\Big(\sum_{i\neq k} n_i r^2\Big)$ to form the individual $A_i^T A_i$ matrices and then $O(dr^2)$ for the Hadamard products, independent of M and N.

A practical preconditioner is then

$$\mathcal{M}_1(X) := KX(G + \lambda I_r), \qquad \text{equivalently} \qquad M_1 := (G + \lambda I_r) \otimes K. \tag{12}$$

This is SPD if K is SPD and $G + \lambda I_r$ is SPD (true for $\lambda > 0$). The motivation is that, for fully observed data, the mode-k normal equations can be transformed (by left-multiplying with K^{-1}) into a system involving $G \otimes K + \lambda I$, so (12) captures the main coupling in r while keeping a single kernel factor K.

Applying $\mathcal{M}_1^{-1}$. To compute $Z = \mathcal{M}_1^{-1}(R)$, we solve

$$KZ(G + \lambda I_r) = R.$$

This can be done by two small linear algebra steps:

1. Solve $KY = R$ for $Y \in \mathbb{R}^{n\times r}$ (i.e. apply K^{-1} to each column).
2. Solve $Z(G + \lambda I_r) = Y$ on the right, i.e. $Z = Y(G + \lambda I_r)^{-1}$.

Step (1) costs $O(n^2 r)$ per application after a Cholesky factorization of K (one-time $O(n^3)$). Step (2) costs $O(nr^2)$ per application after a Cholesky factorization of $G + \lambda I_r$ (one-time $O(r^3)$). Since typically $r \ll n$ and $r \ll q$, this extra $O(nr^2)$ is modest, and the improved conditioning can reduce PCG iterations substantially.

Remark 2 (Alternative: sample-weighted Gram G_Ω). In heavily missing settings, $G = Z^T Z$ may overestimate directions seldom observed. One can instead form

$$G_\Omega := \sum_{t=1}^{q} z^{(t)} (z^{(t)})^T \in \mathbb{R}^{r\times r},$$

which uses only observed rows of Z. This costs $O(qr^2)$ and may better reflect the masked operator. Then (12) can be used with G_Ω in place of G.

7 Complexity analysis (no N and no M)

We assume $n, r < q \ll N$ and $n \ll M$.

7.1 One-time costs per mode-k subproblem

- **Precompute implicit Khatri–Rao rows on observations.** Compute $z^{(t)}$ via (6) for $t = 1, \ldots, q$. This costs
$$O\big(qr(d-1)\big)$$
flops if done directly as a Hadamard product of $(d-1)$ length-r vectors per observation. Memory to store $\{z^{(t)}\}$ is $O(qr)$ if stored; otherwise compute on the fly.
- **Compute $B = TZ$ from observations.** Using accumulation $B(i^{(t)}, :) \mathrel{+}= \tau_t (z^{(t)})^T$, costs $O(qr)$.
- **Compute $C = KB$.** Dense multiplication costs $O(n^2 r)$ (or less if K admits fast multiplication).
- **Preconditioner setup.** For $\mathcal{M}_0$: compute Cholesky of K, cost $O(n^3)$. For $\mathcal{M}_1$: additionally compute G via (11) in $O(\sum_{i\neq k} n_i r^2)$ and Cholesky of $G + \lambda I_r$ in $O(r^3)$.

None of these steps require operations scaling with N or M.

7.2 Per-iteration costs in PCG

Each PCG iteration requires:

1. One application of $\mathcal{A}$ to a search direction P_t.
2. One application of the preconditioner inverse $\mathcal{M}^{-1}$ to the residual.
3. A small number of Frobenius inner products and axpy updates costing $O(nr)$.

Cost of $\mathcal{A}(X)$. Using the matrix-free method:

- $Y = KX$: $O(n^2 r)$.
- Accumulation over observations (8): q dot products and q vector accumulations, each $O(r)$, total $O(qr)$.
- Final multiplication $K(U + \lambda X)$: $O(n^2 r)$.

Hence

$$\text{cost}(\mathcal{A}\text{-apply}) = O(qr + n^2 r). \tag{13}$$

Cost of $\mathcal{M}^{-1}(R)$.

- For $\mathcal{M}_0^{-1}$: solve $KZ = R$ for Z and scale by $1/\lambda$, costing $O(n^2 r)$ per application after Cholesky.
- For $\mathcal{M}_1^{-1}$: solve $KY = R$ ($O(n^2 r)$) and then $Z = Y(G + \lambda I_r)^{-1}$ ($O(nr^2)$), totaling

$$O(n^2 r + nr^2).$$

Thus each PCG iteration costs

$$O(qr + n^2 r) \quad \text{(dominant)}.$$

This should be compared to the direct-solve scaling $O(n^3 r^3)$.

7.3 Iteration count

Let A be the SPD matrix in (1), and let M be the SPD preconditioner matrix. Standard CG theory (applied to $M^{-1/2} A M^{-1/2}$) implies that the error in the A-norm decreases as

$$\frac{\|e_t\|_A}{\|e_0\|_A} \le 2 \left(\frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1} \right)^t, \qquad \kappa := \kappa(M^{-1} A),$$

where $\kappa(\cdot)$ denotes the spectral condition number. Thus the iteration count is

$$t = O\big(\sqrt{\kappa}\, \log(1/\varepsilon)\big).$$

A better preconditioner reduces κ and hence t.

8 Summary

The key to an efficient iterative solve of (1) is:

- Use PCG, which requires only matrix-vector products and preconditioner solves.
- Implement the matrix-vector product *matrix-free* via the operator form $\mathcal{A}(X) = K\mathcal{P}_\Omega(KXZ^T)Z + \lambda KX$.
- Compute $\mathcal{P}_\Omega(KXZ^T)Z$ using only the q observed entries, by looping over observations and using implicit Khatri–Rao rows $z^{(t)}$, never forming Z (size $M \times r$) nor any $n \times M$ dense intermediate.
- Choose an SPD preconditioner such as $\mathcal{M}_0(X) = \lambda KX$ (cheap and robust) or $\mathcal{M}_1(X) = KX(G + \lambda I_r)$ (stronger, still cheap), both applied using precomputed Cholesky factorizations.

Under a dense-kernel model, the resulting per-iteration complexity is $O(qr + n^2 r)$, with no dependence on N and no dependence on M beyond the observed-entry list.

References

[1] M. R. Hestenes and E. Stiefel, Methods of conjugate gradients for solving linear systems, *Journal of Research of the National Bureau of Standards*, 49 (1952), pp. 409–436.

[2] Y. Saad, *Iterative Methods for Sparse Linear Systems*, 2nd ed., SIAM, 2003.

[3] J. R. Shewchuk, An introduction to the conjugate gradient method without the agonizing pain, Technical report, Carnegie Mellon University, 1994.

[4] T. G. Kolda and B. W. Bader, Tensor decompositions and applications, *SIAM Review*, 51(3) (2009), pp. 455–500.

[5] N. Aronszajn, Theory of reproducing kernels, *Transactions of the American Mathematical Society*, 68 (1950), pp. 337–404.

[6] B. Schölkopf and A. J. Smola, *Learning with Kernels*, MIT Press, 2002.